



Notes

May 11, 2026

{ lean-mode }

Invited [Claudiu Leoveanu-Condrei](#) [robert maria](#) [nhigueraruiz@gmail.com](#)
[andrei.reanu@gmail.com](#) ~~[corpaci.luiza@gmail.com](#)~~

Attachments  { lean-mode }

Meeting records  Transcript

Summary

Study session covered function definition patterns and scoping mechanics with repository management workflows and alignment for future.

Function and Type Analysis

Participants explored function definitions, lambda notation, and type inference. They concluded that explicit programming styles are preferred over redundant syntax variations.

Scoping and Proof Tactics

The group analyzed section and namespace keywords for code organization. They confirmed function equivalence via reflexivity proofs to validate compiler behavior.

Workflow and Repository Management

The team aligned on direct repository pushing protocols and assigned leadership for upcoming sessions. They decided to maintain momentum by continuing independent study during the break.

Rate this Summary: [Helpful](#) or [Not Helpful](#)

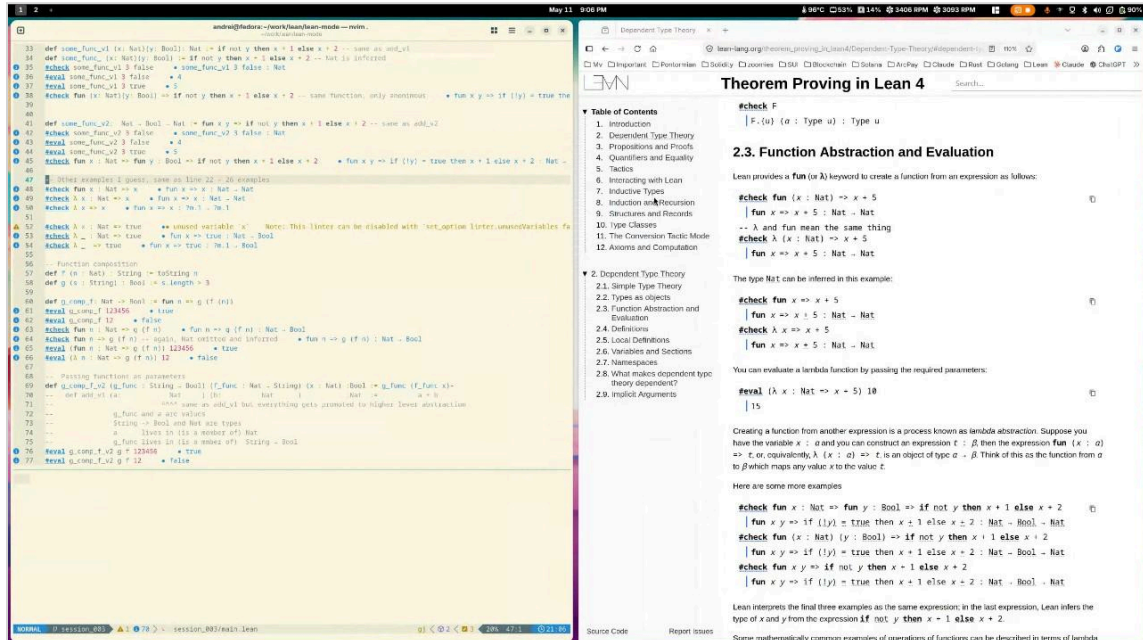
Next steps

- ☐ [Andrei Ungureanu] Share GitHub Handle: Send GitHub username via Discord channel.
- ☐ [Andrei Ungureanu] Review Chapters: Review the remaining chapters of the current section during the 2-week break period.
- ☐ [Claudiu Leoveanu-Condrei] Guide Session: Guide the next learning session, covering the last 2 chapters, scheduled for the 18th.
- ☐ [Claudiu Leoveanu-Condrei] Add Notes: Add the notes from the current meeting call.
- ☐ [Andrei Ungureanu] Push Content: Clean up and push the session files (session 003) to the main branch.
- ☐ [Andrei Ungureanu] Ping Discord: Notify Claudiu via Discord if the attempt to push content fails.

Details

Did the screenshots in this section make your notes [better](#) or [worse](#)?

- **Meeting Introduction and Logistics:** The meeting began with Andrei Ungureanu confirming progress on their guide, having completed two chapters. After addressing technical issues with screen sharing and audio levels, the participants settled into a review of the material ([00:00:00](#)).



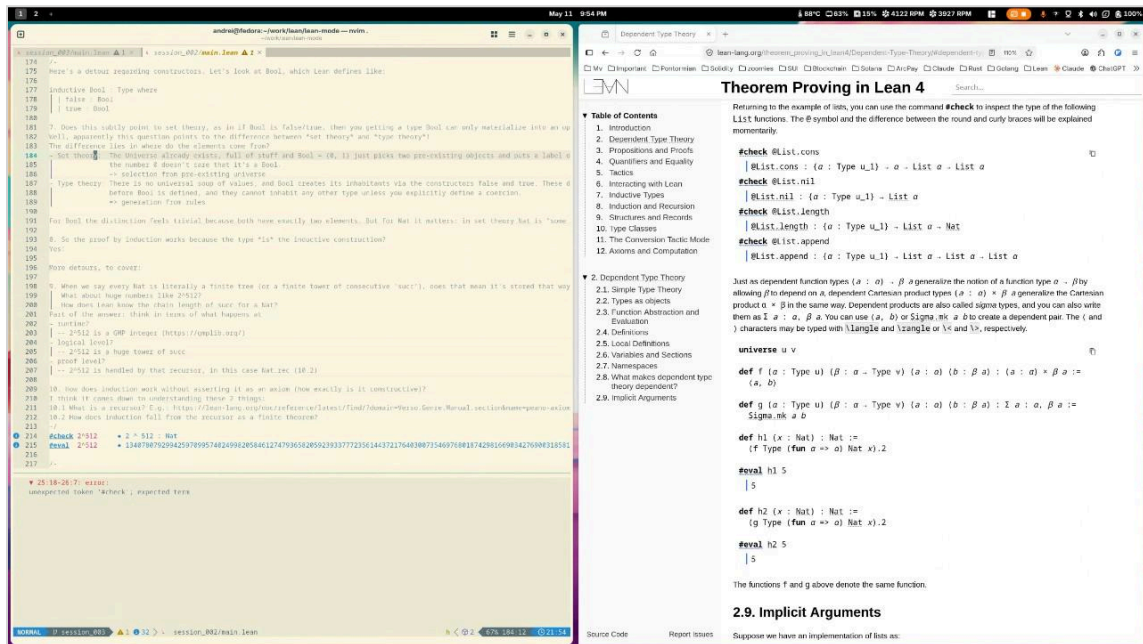
- **Function Definition Overview:** Andrei Ungureanu reviewed the fundamental ways to define functions in the material, noting that they had previously studied different syntaxes, including named parameters with inferred output types and more explicit forms (00:02:07).
- **Currying and Partial Application:** Claudiu Leoveanu-Condrei and Andrei Ungureanu discussed currying, identifying it as a form of partial application. Andrei Ungureanu noted that while this concept initially felt complex, it became straightforward once the functions were deconstructed step by step (00:03:19).
- **Lambda Notation:** The participants discussed the use of lambda as an alias for fun in function definitions. Claudiu Leoveanu-Condrei noted that this notation is likely preserved from lambda calculus (00:04:27). Nelson Higuera Ruiz clarified that lambda functions are generally used to define functions without specific names (00:05:36).
- **Type Inference and Coercion:** Andrei Ungureanu demonstrated how the language infers types based on provided values, such as identifying a natural number versus a float. Claudiu Leoveanu-Condrei observed that there appears to be no automatic coercion between types, which is a feature they both appreciated for the sake of explicit programming (00:06:41).
- **Alternative Function Syntaxes:** The participants explored various ways to define the same function, comparing versions with explicit versus inferred output types. Andrei Ungureanu questioned why the language provides so many options for

defining functions, and Claudiu Leoveanu-Condrei noted the potential for confusion when choosing between them ([00:08:40](#)).

- **Refining Function Definitions:** Claudiu Leoveanu-Condrei and Andrei Ungureanu discussed the apparent redundancy of the various definition styles presented in the book. They concluded that while the repetition serves as subtle reinforcement learning, there is no clear consensus on why one style is preferred over another, leading to a general preference for being explicit as per the principles of Python's philosophy ([00:12:43](#)).
- **Composition Patterns:** Andrei Ungureanu presented examples of function composition, contrasting the more readable but specific approach with a more abstract, verbose version. They noted that the latter allows for the composition of any two functions that match the required input and output types ([00:17:09](#)).
- **Abstracting Functions and Types:** The group reviewed the relationship between values and types, highlighting how variables like `a` and `g-funk` fit into the hierarchy of types. Claudiu Leoveanu-Condrei suggested that these abstractions are useful for handling complex proofs where the user might not know the exact type beforehand ([00:19:43](#)).
- **Higher Level Abstractions:** Andrei Ungureanu and Claudiu Leoveanu-Condrei discussed the hierarchy of types, specifically type zero and type one. They explored how one might perform higher-level abstractions by explicitly defining the levels of the types involved ([00:22:06](#)).
- **Local Variable Scoping:** Andrei Ungureanu demonstrated the use of the `let` keyword for defining locally scoped variables within a function. They explored different formatting options, such as using semicolons versus multi-line definitions to maintain readability ([00:25:10](#)).
- **Shadowing and Variable Constraints:** The participants observed that the language prevents variable shadowing, which would cause an error if a variable name is reused in a nested scope. Claudiu Leoveanu-Condrei remarked that this behavior aligns with the language's lack of automatic coercion ([00:29:04](#)).
- **Type Variables:** Andrei Ungureanu introduced the `variable` keyword for defining multiple type variables. The group clarified that these variables can represent any type, including functions, provided they fall within the defined type hierarchy ([00:30:21](#)).
- **Function Equivalence Proofs:** Andrei Ungureanu and Claudiu Leoveanu-Condrei utilized reflexivity via the RFL tactic to prove the equivalence of different function

definition styles. They confirmed that the compiler treats these different definitions as fundamentally the same ([00:33:31](#)).

- **The Section Keyword:** The group analyzed the section keyword, which acts as a scoping mechanism. While it initially seemed ambiguous, they determined that it serves to organize code and manage local variables, with the internal variables being discarded upon exiting the section ([00:37:10](#)).
- **The Namespace Keyword:** Claudiu Leoveanu-Condrei and Andrei Ungureanu compared sections to namespaces, noting that namespaces function similarly to structures. They observed that namespaces can be opened to access functions without prefixing the namespace name, effectively acting like an import statement ([00:43:39](#)).
- **Universes and Types:** The participants engaged in a deep dive regarding the concept of universes, specifically U and V. Claudiu Leoveanu-Condrei explained that these represent hierarchies of types, and Andrei Ungureanu examined the list as an example of a universal type that can take any alpha type as its element ([00:47:29](#)).



- **Future Logistics:** Andrei Ungureanu requested to share their GitHub handle in the Discord channel so they could push their code directly to the repository. Regarding upcoming meetings, Andrei Ungureanu mentioned a two-week holiday. The participants agreed that Andrei Ungureanu would review the remaining chapters independently during this time, while Claudiu Leoveanu-Condrei and

Nelson Higuera Ruiz would continue with the final chapter to maintain momentum, with the group resuming sessions together afterward ([00:54:31](#)).

- **Next Meeting Schedule and Leadership:** Claudiu Leoveanu-Condrei will serve as the guide for the upcoming session covering the final two chapters, which is scheduled for the 18th. Andrei Ungureanu intends to join the session despite it being their travel day for flights, noting that they plan to use headphones while at the airport ([00:57:23](#)).
- **GitHub Repository Access:** Claudiu Leoveanu-Condrei invited Andrei Ungureanu to the repository via an email notification ([00:57:23](#)). Although there was initial uncertainty regarding the configuration of access rights and the visibility of the settings tab, Claudiu Leoveanu-Condrei concluded that Andrei Ungureanu should possess the necessary permissions, as their access status appears to be automatically inferred in a manner similar to that of Nelson Higuera Ruiz ([00:59:47](#)).
- **Repository Contribution Workflow:** Participants clarified the protocol for updating the repository, determining that they should push directly to the main branch rather than creating separate branches. Claudiu Leoveanu-Condrei committed to adding the notes from the current meeting to the repository, while Andrei Ungureanu will add their specific readme files ([01:01:10](#)).
- **Finalizing Updates and Meeting Conclusion:** Andrei Ungureanu planned to refine their work and push it to the repository within 10 minutes. If they encounter any technical issues during this process, they will reach out to Claudiu Leoveanu-Condrei on Discord for assistance. The participants then concluded the meeting ([01:02:08](#)).

You should review Gemini's notes to make sure they're accurate. [Get tips and learn how Gemini takes notes](#)

How is the quality of **these specific notes**? [Take a short survey](#) to let us know your feedback, including how helpful the notes were for your needs.



Transcript

May 11, 2026

{ lean-mode } - Transcript

00:00:00

Claudiu Leoveanu-Condrei: Oh, so uh Andre, I hope it was not too short on a too short notice about the guide. I the reason

Andrei Ungureanu: Uh no, I had already done uh like two chapters or something.

Claudiu Leoveanu-Condrei: I Ah,

Andrei Ungureanu: So it was it

Claudiu Leoveanu-Condrei: okay. Cool. Because this week this weekend I I was gone uh to some uh

Andrei Ungureanu: wasn't

Claudiu Leoveanu-Condrei: to some friends because it was my birthday and I Yeah,

Andrei Ungureanu: Oh,

Claudiu Leoveanu-Condrei: we we had Thank you.

Andrei Ungureanu: happy birthday

Claudiu Leoveanu-Condrei: And uh yeah, that's why I had to be gone and I could not uh spend as much time as the other the previous time on it. Okay. Uh yeah, please.

Andrei Ungureanu: Oh yes, I'll share my screen

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: now.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: Everything okay? see it

Claudiu Leoveanu-Condrei: Yes,

Andrei Ungureanu: right.

Claudiu Leoveanu-Condrei: it's still buffering on my

Nelson Higuera Ruiz: I can see.

Claudiu Leoveanu-Condrei: end.

Andrei Ungureanu: Uh yeah,

Nelson Higuera Ruiz: Well,

Andrei Ungureanu: for some reason uh you got you both got uh cut off on me.

00:01:10

Andrei Ungureanu: I'm not sure why. Do you hear me?

Claudiu Leoveanu-Condrei: Yes,

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: but now your screen uh your screen got kicked out.

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: But we can hear right

Nelson Higuera Ruiz: Yeah, I can. I mean, I I I put it a little bit more volume though because it's a bit low,

Claudiu Leoveanu-Condrei: Nelson.

Nelson Higuera Ruiz: but but I can hear you.

Andrei Ungureanu: Can you see now? I shared it my screen again.

Nelson Higuera Ruiz: Yes.

Claudiu Leoveanu-Condrei: Yes.

Andrei Ungureanu: Okay, perfect.

Claudiu Leoveanu-Condrei: Are you using Hyperland? That's Hyperland, right?

Andrei Ungureanu: Uh using Fedora so I think which is using hyperl. So you mean about the tiling or what?

Claudiu Leoveanu-Condrei: Ah, okay. Yeah, the timing. Yes, it's it's hyperola,

Andrei Ungureanu: Uh, it's an add-on,

Claudiu Leoveanu-Condrei: right?

Andrei Ungureanu: I think. Uh, extension, how do they call it? It's tiling shell. This stuff here. Uh,

Claudiu Leoveanu-Condrei: Ah, okay.

Andrei Ungureanu: I'm not sure what it is.

Claudiu Leoveanu-Condrei: Got it.

00:02:07

Claudiu Leoveanu-Condrei: A tiling shell. Okay. Okay. Probably Fedora has a different one. Sorry.

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: It's uh anyway it looks very similar to uh to to

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: hyperl.

Andrei Ungureanu: You you can set up uh custom layout. So this these here are all done by me.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: So according to my needs um

Claudiu Leoveanu-Condrei: Okay, fair enough.

Andrei Ungureanu: yeah uh yeah so uh this

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: uh four or five chapters uh they basically deal with functions mostly I mean uh it's all some stuff even overlap

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: um in various chapters.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: So before starting I actually did a small uh recap uh on what uh was in the previous two chapters. So this is like the main takeaway I think. So the way you uh define functions. So you have this uh named uh param version uh which can also um uh not have the output net because it's inferred uh you also have this um

00:03:19

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: more explicit form uh which I took a bit of uh you know the structuring to it to um like figure out this stuff

Claudiu Leoveanu-Condrei: Mhm.

Nelson Higuera Ruiz: Excuse

Andrei Ungureanu: which This is was like first thing that um was to me to have this uh function that's derived from this function. So you see my cursor, right? Uh yeah.

Nelson Higuera Ruiz: me.

Claudiu Leoveanu-Condrei: Yes.

Andrei Ungureanu: So this is was like the the first uh thing that twisted my brain a bit. Uh but it made sense at some point. Uh cuz you basically have the three baked in like this. So if you dstructure it and you like u go through it step by step, it's pretty straightforward eventually. So um

Claudiu Leoveanu-Condrei: So, so this is this is basically the curring right is the um is the partial

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: application.

Andrei Ungureanu: Yes.

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: So curring uh I don't know you are familiar with curring Nelson.

Andrei Ungureanu: Oh

Nelson Higuera Ruiz: Yes.

Claudiu Leoveanu-Condrei: Yeah.

00:04:27

Claudiu Leoveanu-Condrei: So it's that basically literally

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: that.

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: Yeah. Yeah. I think I mentioned the uh terminology some somewhere, but I didn't write it here. Um anyway,

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: uh so uh the 2.3 chapter starts with pretty much the same stuff here. Uh the new thing here is that for some reason they use uh lambda as an alias for for uh

Claudiu Leoveanu-Condrei: London.

Andrei Ungureanu: fun. Uh not sure why.

Claudiu Leoveanu-Condrei: Yes.

Andrei Ungureanu: Uh so basically they're interchangeable. Even if you type uh backwards slash and uh fun you get lambda. So

Claudiu Leoveanu-Condrei: It's because of the λ I mean the first functional rights is it's it's basically

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: lambda calculus and they they preserve the λ I assume I don't know uh I actually didn't didn't read this but lambda calculus was the first calculus and

Nelson Higuera Ruiz: Yeah.

Claudiu Leoveanu-Condrei: it's uh yeah I mean probably that's why they just preserve the the notation because the lambda is a function

00:05:36

Nelson Higuera Ruiz: In general, uh, lambda function is whenever you want to write a function but without a name. So you can use

Andrei Ungureanu: Yeah, that's what I thought too because the other languages have the same uh terminology.

Nelson Higuera Ruiz: Yes, indeed.

Andrei Ungureanu: So yeah,

Nelson Higuera Ruiz: Indeed. Yeah.

Andrei Ungureanu: somehow it made sense to them. Yeah. Uh so yeah,

Nelson Higuera Ruiz: Yeah.

Andrei Ungureanu: this is pretty much this again.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Uh this doesn't uh have the explicit output uh type. Uh this is fine. This is lambda again. Yeah.

Claudiu Leoveanu-Condrei: So out of curiosity here like how how is it inferred because that's that's curious to me. So you wrote there that not is inferred. No, no. In the in the bottom one,

Andrei Ungureanu: this this one

Claudiu Leoveanu-Condrei: uh, line 25 because that's like lambda

Andrei Ungureanu: you have the five and you can't when you add something to n you get you can

Claudiu Leoveanu-Condrei: x.

Andrei Ungureanu: only add the net to it so it's uh the output is n that's how A

00:06:41

Claudiu Leoveanu-Condrei: Yeah, but that's that's not true, right? Because what if you add minus one?

Andrei Ungureanu: hit

Claudiu Leoveanu-Condrei: You can add integers to a not. Ah, no. Sorry, you cannot add but but of course you cannot but like how does it infer that is a not

Andrei Ungureanu: cuz five is a nut.

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: That's how I know because if

Claudiu Leoveanu-Condrei: ah okay so I see I see so basically they they make a distinction

Andrei Ungureanu: you

Claudiu Leoveanu-Condrei: so can you put like 5.0 zero there.

Andrei Ungureanu: just see if it's a float. Yeah, probably won't work. No. Oh,

Claudiu Leoveanu-Condrei: Ah,

Andrei Ungureanu: it becomes a float.

Claudiu Leoveanu-Condrei: that's smart.

Andrei Ungureanu: Yeah,

Nelson Higuera Ruiz: Yeah.

Andrei Ungureanu: right.

Claudiu Leoveanu-Condrei: That's very smart. Nice.

Nelson Higuera Ruiz: Yeah.

Claudiu Leoveanu-Condrei: Very nice.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: And if you write pi, can you put a slash pi?

Andrei Ungureanu: Hold on. Sorry.

Claudiu Leoveanu-Condrei: Uh net is inferred. Okay. So it doesn't know that is pi.

00:07:43

Claudiu Leoveanu-Condrei: Okay. Yeah. Fair enough.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: I was curious. Okay. Yeah. Okay. Very very cool. Yeah. See the power of types because you cannot Yeah. It can x can only be matt. That's so cool.

Andrei Ungureanu: The thing is you probably have to be very explicit of how you combine types. So uh it's not like another programming languages when you like if you add a float and the net you get a float. So this doesn't uh uh explicitly

Claudiu Leoveanu-Condrei: Mhm. This doesn't allow it.

Nelson Higuera Ruiz: All right.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: It's so basically there's no coercion in this language then at least so far as so far as

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: we learn it there's no coercion happening which is honestly I like that you have to be explicit about the operation.

Andrei Ungureanu: Exactly.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Cool. And all those things are I suppose they are the same thing right?

00:08:40

Andrei Ungureanu: Yeah. Yeah.

Claudiu Leoveanu-Condrei: Uh yeah.

Andrei Ungureanu: They're exactly this $x + 5$ and written in all these

Claudiu Leoveanu-Condrei: So

Andrei Ungureanu: forms here. Uh yeah.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: So this is pretty much the same thing. So kind of like what was happening here I guess right you have uh function and you give it a uh

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: value and it's it's baked in basically but this is just a value it's not a yeah it's not returning defining a function. Uh yeah so what have I written here in this case it's a value yeah it's not takes a not to not basically here uh so this is pretty much again the same thing like

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: I also have written here. So it's like uh the function on line two but it has another uh it's defined differently. So again it's the same thing. Uh this here has uh again the same thing without having the output where is inferred. Uh some more. Um

00:10:00

Claudiu Leoveanu-Condrei: Just one one quick question from my side. So line so both lines 33 and 34.

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: So is that like currying or so so what happens like if you remove because there are two sets of parenthesis right the x in in parenthesis and the y. So is there a difference?

Andrei Ungureanu: Right.

Claudiu Leoveanu-Condrei: So is this different than x uh not then you put a comma instead of parenthesis. So I suppose that's a topple,

Andrei Ungureanu: Like here

Claudiu Leoveanu-Condrei: right? No, no, no.

Andrei Ungureanu: here.

Claudiu Leoveanu-Condrei: Uh you you let the the first parenthesis you put a comma

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: there.

Andrei Ungureanu: here between the uh we were sorry.

Claudiu Leoveanu-Condrei: Uh after the Yeah, there. Yeah, exactly. You delete those two parentheses. Yeah, but without the parenthesis.

Andrei Ungureanu: Yes.

Claudiu Leoveanu-Condrei: Yeah, like that. Yeah. Okay. So, that's not possible.

Andrei Ungureanu: Uh

Claudiu Leoveanu-Condrei: Probably because of how it is defined afterwards.

Andrei Ungureanu: well uh that's

Claudiu Leoveanu-Condrei: That's a to that's a topple I believe.

00:11:05

Andrei Ungureanu: yes but I yeah it I don't think this is the right uh way the signature basically right to um

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: But we had something like that in the other one.

Andrei Ungureanu: write in uh 2.2

Claudiu Leoveanu-Condrei: I remember that probably h ah actually we we did not sorry because those were not function we didn't define anything there there was just like uh

Andrei Ungureanu: tool.

Claudiu Leoveanu-Condrei: types okay fair enough yeah no there was not there was not it was just a type uh here is a function definition

Andrei Ungureanu: Yeah. Um yeah. So this is basically checking um some values

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: on the function. Um again same function. Oh yeah this is like the same function as the uh one here but it's in this anonymous form. Um again this is the other form which we had in at actually for me uh I mean you you can define a function in all these ways in all these ways uh I'm guessing at some point you need one uh type or the other right because uh I mean it makes sense in sort of uh having as much options as possible, but uh they probably will uh count at some point, right?

00:12:43

Andrei Ungureanu: That that's what was my question for you. Actually, Nson, maybe you have more experience uh with this stuff.

Nelson Higuera Ruiz: No, not

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: No.

Nelson Higuera Ruiz: really.

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: C can you repeat the question please? Don't think I understood

Andrei Ungureanu: Yeah. Sorry, I was very upset. Uh I I don't really see at this point uh why do we have all these

Claudiu Leoveanu-Condrei: it.

Andrei Ungureanu: options for defining a function? Um it's it it seems

Claudiu Leoveanu-Condrei: Ah okay. Yeah. I mean, we discussed this the last time. We discussed this the last time.

Andrei Ungureanu: superfluous.

Claudiu Leoveanu-Condrei: Uh I don't understand either like why why would you go like with with one over the other? I I also don't get it because for me was confusing as well. If you remember we had four definitions last time of lambda functions. So like we which one am I am I supposed to choose?

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Uh I don't know if there is a convention or something.

00:13:39

Claudiu Leoveanu-Condrei: probably yes as as the way you define variables like you know it's um or or type variables I suppose there is some um yeah there is some form of convention of how do you use the lambda function I I would assume but

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: I don't think there is any particular reason of one over the I mean from the syntax I just don't see it I I like one syntax more over the other but Other than that, there's no I don't know

Andrei Ungureanu: Yeah. Yeah. I forgot we talked about that. But yeah.

Claudiu Leoveanu-Condrei: why

Andrei Ungureanu: Um and actually these chapters uh like repeat some information uh regarding this and again um considering all of this it seems a bit weirdly structured uh for me but um I don't know maybe it will make some sense uh at some point through going through the book

Claudiu Leoveanu-Condrei: So I I take the I take the repetition as being uh probably

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: Oh

Claudiu Leoveanu-Condrei: uh subtle reinforcing,

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: you know.

Andrei Ungureanu: Yeah.

00:14:51

Andrei Ungureanu: Reinforcement learning.

Claudiu Leoveanu-Condrei: So they just Yeah, exactly.

Andrei Ungureanu: Right.

Claudiu Leoveanu-Condrei: So they they're just telling you that ah be pay attention to this one.

Andrei Ungureanu: Mhm. Right. Uh yeah. So again more example like this. Uh I don't know what there is to talk about here. Uh again fun and lambda interchangeable. This has the output type. This here doesn't. Uh this uh since it's a constant function x doesn't really matter. So it's uh it a warning not an error. If you put underscore uh `lin` doesn't complain anymore. Um,

Claudiu Leoveanu-Condrei: I mean about for instance the type itself type inference

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: or I think that's useful because there are for instance that there there could be proofs in which you just don't know the type. So I would assume that you have some weird ways in which you use `sorry` and `um` and other keywords and then you let the I mean if you have if you have like a complex

Andrei Ungureanu: Mhm.

00:16:09

Claudiu Leoveanu-Condrei: proof and then you don't infer the type then then Lynn should do it for you right so at least in some cases you might not know the type

because it's so convoluted and complicated the proof that you're writing But then the lean autocomp complete it for you and you and you just check it like okay so what's the type now is it this or that and I I assume I guess that's useful I mean I would assume that you you will probably not know at some point you make some educated guesses when you make a proof you you might not be sure that that should be the type or not but you let it without the type and force lean to um materialize the type for you that could be a is why you sometimes don't put a

Andrei Ungureanu: Right. But I remember reading uh somewhere in these chapters that uh

Claudiu Leoveanu-Condrei: type.

Andrei Ungureanu: um you should always uh add the type even though it is inferred. It is preferred for you to add the

Claudiu Leoveanu-Condrei: I mean, yes, it even even in Python,

00:17:09

Andrei Ungureanu: type.

Claudiu Leoveanu-Condrei: the Zen of Python says always be explicit. Explicit instead of implicit.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: I I I like that so much. I agree. Yeah, you should be

Andrei Ungureanu: Mhm. Right. Uh yeah,

Claudiu Leoveanu-Condrei: explicit.

Andrei Ungureanu: so this is when uh things got a bit more interesting and uh actually somewhere here it clicked for me like going through for uh to the high level abstractions. So you got uh simple uh composition example here. got the two functions here uh which you and you can define the composition uh in various ways like this is the easiest way I guess but uh you always have to refer to the types uh you have defined for f and g so it's not uh it's it's a bit more complicated to read uh if you have it like this uh but

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: uh Then the more explicit form which is actually this one. Um so this is more like yeah this is like the most

Claudiu Leoveanu-Condrei: More verbose.

00:18:18

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: verbose uh option there is and these are all based on the uh definition here which is uh but actually this is um not quite the same thing because you define here doc this version you have uh the these two uh specific uh specifically functions right because you use f and g here when you define uh g comp f so this is uh

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: valid only for these two f and g functions if you look at this one the

Claudiu Leoveanu-Condrei: That's

Andrei Ungureanu: passing yeah because you can have any function that takes a string and returns a bull

Claudiu Leoveanu-Condrei: abstract.

Andrei Ungureanu: here right not just uh g so the same with f uh

Claudiu Leoveanu-Condrei: Yeah. So this is like a this is like a composition pattern over functions

Andrei Ungureanu: here. Yeah. Yeah. Exactly. And this is when it clicked for me cuz you take the uh

Claudiu Leoveanu-Condrei: themselves.

Andrei Ungureanu: ad example. So this is like the the correspondence of the two. Uh so uh like if you uh go through each of these two function uh function definitions you get in the first example.

00:19:43

Andrei Ungureanu: So these are like uh uh they correspond to each other on the column. So g funk and a are values here and uh string to bull

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: and net are types uh and this is where each of them lives. So a is a member of net and obviously g funk is a member of uh string to bull and that's how I uh and it's quite

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: clear here uh if you like uh I really like this picture I think it's like uh what defines the whole or at least uh this part of uh bl we studied up until now.

Claudiu Leoveanu-Condrei: Th this should this should have probably answer you also like the when we discussed the thing about the capital

Andrei Ungureanu: Um,

Claudiu Leoveanu-Condrei: type. Do you remember like when we had abstractions of types?

Andrei Ungureanu: type one, type two zero, type, right? That's what you're talking.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Yeah. Yeah. Yeah. I know. Yeah. This true cuz we we'll go through even the higher level here.

00:20:44

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: So, uh yeah, this is exactly that.

Claudiu Leoveanu-Condrei: Yes.

Andrei Ungureanu: Uh right. So uh now that we have this uh more uh general type of composition yeah I've had G and F I didn't bother to give it new functions but uh it surely works I mean new G's or new F's uh but this this this one uh this one work uh and you got also this uh kind of uh again another way to write the the the same fun the same composition function Basically, uh like we also have with the um adding

Claudiu Leoveanu-Condrei: Yeah. So that that that's very nice because you're basically removing the

Andrei Ungureanu: um

Claudiu Leoveanu-Condrei: um in the line 78 you're no longer carrying the name of the function in the input space you're just using it in the output space right

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: because the that you're just using the mapping itself without giving it any name right so in the first one it was gunk you name the mapping you were explicit in the in the

Andrei Ungureanu: Right.

Claudiu Leoveanu-Condrei: one you're just saying this is a general mapping whatever it may be called from string to bool yeah exactly yeah

00:22:06

Andrei Ungureanu: Yeah. And this is a lambda uh right again here.

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: that's

Andrei Ungureanu: So it's a lambda function. Yeah.

Claudiu Leoveanu-Condrei: lambda

Andrei Ungureanu: So uh yeah, this basically proves all the stuff here. Proves that this works. uh and this is what we talked about a bit earlier.

Claudiu Leoveanu-Condrei: mhm

Andrei Ungureanu: So this when you go to the higher level abstraction you also define the types. So uh you also need to define the types and uh

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: like you said uh alphabet gamma live in are type of zero that live in type one and probably you like surely you can do even higher level abstraction if you know what lives in type

Claudiu Leoveanu-Condrei: I I guess I guess if you put their type one right

Andrei Ungureanu: one.

Claudiu Leoveanu-Condrei: afterwards.

Andrei Ungureanu: Yeah, you have to put here,

Claudiu Leoveanu-Condrei: So if you put for alpha No,

Andrei Ungureanu: right? Type somewhere here.

Claudiu Leoveanu-Condrei: I mean uh No,

Andrei Ungureanu: That's what

Claudiu Leoveanu-Condrei: if you put uh the alpha is type one, right?

00:23:02

Claudiu Leoveanu-Condrei: And beta type one and y type one. Yeah.

Andrei Ungureanu: here.

Claudiu Leoveanu-Condrei: After uh space type type space one. Uh no,

Andrei Ungureanu: Oh,

Claudiu Leoveanu-Condrei: it's type space one.

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: Yeah, like that. And then the others if you put then the others again as type one

Andrei Ungureanu: This this here

Claudiu Leoveanu-Condrei: I yeah this is a higher obstruction even more right now.

Andrei Ungureanu: Yeah,

Claudiu Leoveanu-Condrei: Yeah,

Andrei Ungureanu: but you you have could define the type zero uh afterwards,

Claudiu Leoveanu-Condrei: there

Andrei Ungureanu: right? So you you need more than just uh changing type which is actually type zero to type one to make this work, right?

Claudiu Leoveanu-Condrei: doesn't it obstructs

Andrei Ungureanu: I don't

Claudiu Leoveanu-Condrei: automatically?

Andrei Ungureanu: know.

Claudiu Leoveanu-Condrei: Yeah. No, it doesn't.

Andrei Ungureanu: Yeah, but uh here not is not a type not a

Claudiu Leoveanu-Condrei: Ah, it's not a type.

Andrei Ungureanu: type one here.

Claudiu Leoveanu-Condrei: It's it's not a type one. Yeah, that that has to be type basically there.

Andrei Ungureanu: So,

00:24:08

Claudiu Leoveanu-Condrei: All of those has to be type, right?

Andrei Ungureanu: right. But uh can you do a

Claudiu Leoveanu-Condrei: But then the application doesn't make any sense. No, no, of course, because we don't have a type one defined anywhere.

Nelson Higuera Ruiz: Yeah, there's the

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: But I assume this is the pattern, right?

Nelson Higuera Ruiz: product.

Claudiu Leoveanu-Condrei: If you want to abstract it away, you just add the type uh integer. And if you want the universal type, then you just type U to make like the most abstract function possible,

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: the the most abstract composition across all the hierarchies of types.

Andrei Ungureanu: the highest level.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: Okay, cool.

Andrei Ungureanu: Right. So again this like proves that uh this version works. Uh you have another way of writing of course which this one uh all right this just uh puts in order to have only one parenthesis here with just uh to not have them spread out.

Claudiu Leoveanu-Condrei: So they are all types.

Andrei Ungureanu: Yeah.

00:25:10

Andrei Ungureanu: Yeah. Uh same thing.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Uh right. So this was 2.3.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: Uh now 2.4 again a bit of uh reinforcement learning. Uh yeah it's the same stuff basically. You define a double which doubles the this is the function definition doubles the number. uh again you write it uh as a lambda function. This uh also has the output type.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: Uh this is uh basically defines a constant

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: here which you can use it like you would have a number there.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: So uh and this doesn't work.

Claudiu Leoveanu-Condrei: Mhm.

Andrei Ungureanu: Uh oh right it's another function I had somewhere. Uh it it yeah it was in the text so I put it here but it

Nelson Higuera Ruiz: below.

Andrei Ungureanu: definitely

Claudiu Leoveanu-Condrei: Does it work if you put like uh three dot dot dot dot after the pi?

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: I don't remember where I've seen that. So if you put the pi and then you just put at the end of the number at the end of the float if

00:26:23

Andrei Ungureanu: what what

Claudiu Leoveanu-Condrei: you put uh Yeah.

Andrei Ungureanu: here like to

Claudiu Leoveanu-Condrei: If you put like three dots, two dots.

Andrei Ungureanu: for to infer the like all the other

Claudiu Leoveanu-Condrei: Yeah. And if you put two dots.

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: Yeah. No. Okay. It's not. Uh maybe there is something. I don't I think I saw something like that or maybe I'm just like confused. Anyway, sorry.

Andrei Ungureanu: yeah no uh yeah so you can have a float output here just double the the pi float number. Uh this is again sort of composition but it's yeah it's just with an F uh with the F function.

Claudiu Leoveanu-Condrei: composition.

Andrei Ungureanu: Uh yeah so this is just some basic stuff again. Um yeah, this is again uh uh it goes to the like the concrete uh stuff you would do probably uh writing blink code. So at some point you would need the uh local scoped variable. So you just define with the let keyword here. So this uh doesn't exist anywhere else um except on this line uh it's scoped inside the function here.

00:27:56

Andrei Ungureanu: Basically you can also it's it uses uh this here right to write it on the same line. I forgot the name in English of this what's it

Claudiu Leoveanu-Condrei: scoping. Right. Ah, that's a

Andrei Ungureanu: called? Small right? Yeah.

Claudiu Leoveanu-Condrei: semicolon.

Andrei Ungureanu: uh and you can uh write it like this on two lines. Uh it's more readable I guess uh and it doesn't use the

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: semicolon here the same thing. Uh and here you can

Claudiu Leoveanu-Condrei: And And is And is the closure.

Andrei Ungureanu: also

Claudiu Leoveanu-Condrei: So is there closure on y? So like basically if you leave that y should not exist right only in the scope of the function.

Andrei Ungureanu: yeah you mean here right if you write the val uh

Claudiu Leoveanu-Condrei: Okay.

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: something y + 3 something unknown y

Nelson Higuera Ruiz: Yep. Uhhuh.

Claudiu Leoveanu-Condrei: Yeah. Okay.

Nelson Higuera Ruiz: Good. Good.

Claudiu Leoveanu-Condrei: Good.

00:29:04

Claudiu Leoveanu-Condrei: True closure.

Andrei Ungureanu: Yeah. Uh right in here. Uh can you replace it there with the net? Yeah. You define a

Claudiu Leoveanu-Condrei: I I think you left uh you you left the Yeah,

Andrei Ungureanu: type.

Claudiu Leoveanu-Condrei: there. Yeah,

Andrei Ungureanu: Uh so this you define uh a type uh in a in variable a and uh no but yeah

Claudiu Leoveanu-Condrei: the scope.

Andrei Ungureanu: it's again it's called but you can also have types uh in variable right um and here yeah you cannot have this because uh here what happens you have an a here defined right and then you Got another A here that shadows the the first

Claudiu Leoveanu-Condrei: Mhm. The first

Andrei Ungureanu: A. Yeah.

Claudiu Leoveanu-Condrei: one fail

Andrei Ungureanu: And uh this fails. This is the it

Claudiu Leoveanu-Condrei: to synthesize instance of type class synthesize.

Andrei Ungureanu: Yeah. Doesn't

Claudiu Leoveanu-Condrei: That's a very a very uh unique

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: wording.

Andrei Ungureanu: Uh Yeah.

Claudiu Leoveanu-Condrei: I mean that makes sense that so basically shadowing is not allowed.

00:30:21

Claudiu Leoveanu-Condrei: Uh the thing above we seen with the type uh so coercion is not allowed. Shadowing is not allowed so far.

Andrei Ungureanu: Mhm. Exactly. Uh right. So 2.6 just uh introduces this uh variable keyword.

Claudiu Leoveanu-Condrei: variable.

Andrei Ungureanu: Yeah. So you can define uh anything

Claudiu Leoveanu-Condrei: A type variable.

Andrei Ungureanu: basically sorry

Claudiu Leoveanu-Condrei: That's a type variable, right?

Andrei Ungureanu: uh multiple.

Claudiu Leoveanu-Condrei: Yeah. Multiple type variables. Yeah.

Nelson Higuera Ruiz: Right.

Claudiu Leoveanu-Condrei: But they they are basically type.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: I mean it's not like a it can be any natural number right so I mean in that case it's just a type right so it it that's type zero so it has to be like boolean right or string or something like that it can be I assume a a or not even that so type

Andrei Ungureanu: Uh no it's uh like it can

Claudiu Leoveanu-Condrei: wait

Andrei Ungureanu: be even like not to string or not to not right.

Nelson Higuera Ruiz: Thank

Claudiu Leoveanu-Condrei: Ah, so it can be mapping too.

00:31:33

Claudiu Leoveanu-Condrei: Yes. Yes. Okay. Yeah.

Nelson Higuera Ruiz: you.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: But it has to so it cannot be a higher it cannot be more than boolean n and mapping. I think you of course you cannot do more than than the the defined type.

Andrei Ungureanu: Uh yeah, like I you can have type one or that what you're saying or what?

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: I don't know.

Claudiu Leoveanu-Condrei: Yeah. Yeah.

Andrei Ungureanu: Um

Claudiu Leoveanu-Condrei: because notice type one,

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: right?

Andrei Ungureanu: uh type one is

Claudiu Leoveanu-Condrei: So that's what I remembered. Can Can you scroll?

Andrei Ungureanu: uh higher level abstraction than type zero.

Claudiu Leoveanu-Condrei: Can you Yeah.

Andrei Ungureanu: So it

Claudiu Leoveanu-Condrei: So if you go to to the book there you're if you scroll down I think

Andrei Ungureanu: no

Claudiu Leoveanu-Condrei: bit no sorry up wait which was the last one that no no on the book on the right uh on the of the page I think there there was that that table they showed

00:32:26

Andrei Ungureanu: ah the book right

Claudiu Leoveanu-Condrei: us you remember with the hierarchies of types I think it it

Andrei Ungureanu: uh chapter 2.2 or Oh right here. Yeah. Yeah. Yeah.

Claudiu Leoveanu-Condrei: was yeah so uh yeah that was the one it

Andrei Ungureanu: Here it is.

Claudiu Leoveanu-Condrei: was a bit more

Andrei Ungureanu: Yeah. Yeah. Yeah. This

Claudiu Leoveanu-Condrei: this one. Yeah. So, okay. So,

Andrei Ungureanu: one.

Claudiu Leoveanu-Condrei: type is bool. Okay. So, type is bool is not is is is all of that and prop is basically the one below which is just u yeah true or false. Okay, fair enough.

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: I I forgot that it's called prop.

Andrei Ungureanu: Excellent.

Claudiu Leoveanu-Condrei: Uh yeah.

Andrei Ungureanu: I so forgot about this. So this is type one I had

Claudiu Leoveanu-Condrei: Yeah. That that's why you Yeah.

Andrei Ungureanu: like

Claudiu Leoveanu-Condrei: I don't think you can do mappings

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: there.

Andrei Ungureanu: Um yeah, I'm going to try it.

00:33:31

Andrei Ungureanu: But uh basically this is what this chapter says.

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: So you can again it's like chapter before if you write uh if you define a function like this you have to like go what is g so you go with uh upwards and you look at g and what is g right and it's β and you have to go to look at β

Claudiu Leoveanu-Condrei: Oh, that's so Yeah.

Andrei Ungureanu: it's it

Claudiu Leoveanu-Condrei: So, yeah,

Andrei Ungureanu: cumbersome.

Claudiu Leoveanu-Condrei: that's like uh nested. That That's why I hate inheritance. That's basically inheritance in a nutshell. You always have to go and look what what is this class?

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: What about this class? Let's go

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: deeper.

Andrei Ungureanu: Uh yeah. So uh this is like the uh full definition which has all the types. I wrote it because I wanted to uh do a bit of reinforcement learning myself here I guess. Uh so this this uh work with both uh

Claudiu Leoveanu-Condrei: is the same thing.

00:34:34

Andrei Ungureanu: the with both definitions.

Claudiu Leoveanu-Condrei: They are the

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: same.

Andrei Ungureanu: Um so this is another composition which is again the same thing but it uh has the h function.

Claudiu Leoveanu-Condrei: But but I am curious are they really the same?

Andrei Ungureanu: Uh

Claudiu Leoveanu-Condrei: So if you put like uh eval and you let compose equal to compose full and you do it by um the reflection. you remember that strategy to I'm curious if they are really the

Andrei Ungureanu: yeah.

Claudiu Leoveanu-Condrei: same because yeah based on the output they are the same but are they really the same?

Andrei Ungureanu: So meaning like this execute for

Claudiu Leoveanu-Condrei: Yeah, I think it's equal there. So execute twice equal execute twice full and then it's like that

Andrei Ungureanu: here.

Claudiu Leoveanu-Condrei: uh colon equal right uh

Andrei Ungureanu: Uh and

Claudiu Leoveanu-Condrei: and RFL I think no

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: not RFL uh reflect how the hell it was called

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: RFL. Ah, it's actually fell. Okay. Probably autocomplete is

00:35:49

Andrei Ungureanu: RFL.

Claudiu Leoveanu-Condrei: not is not that the syntax I don't think is

Andrei Ungureanu: Uh I'm not sure

Claudiu Leoveanu-Condrei: that ah it's example sorry not not

Andrei Ungureanu: if Yeah.

Claudiu Leoveanu-Condrei: eval. It's example.

Andrei Ungureanu: So it's without the

Claudiu Leoveanu-Condrei: Yeah. Yeah. It's it's it's it's example and then

Andrei Ungureanu: hash.

Claudiu Leoveanu-Condrei: column. Uh no no it's okay there. It's example and then column. After example there is a column.

Andrei Ungureanu: here, I

Claudiu Leoveanu-Condrei: Yeah. Yeah. No it is it is the same. Yeah.

Andrei Ungureanu: guess.

Claudiu Leoveanu-Condrei: Okay. So basically Nelson what that does is is using uh reflexivity in mathematics to prove that the parts are the same.

Nelson Higuera Ruiz: I see it. I

Claudiu Leoveanu-Condrei: So it's a tactic. So you you know reflexivity in logic.

Nelson Higuera Ruiz: think

Claudiu Leoveanu-Condrei: So x equals y equals x is like you just prove those things are the same

Andrei Ungureanu: It's here I think.

Claudiu Leoveanu-Condrei: thing.

Nelson Higuera Ruiz: anyway.

00:37:10

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Yeah. Okay.

Andrei Ungureanu: Um

Claudiu Leoveanu-Condrei: They so they are truly the same. Okay.

Andrei Ungureanu: yeah. Uh and another composition of composition.

Claudiu Leoveanu-Condrei: Nice.

Andrei Ungureanu: Uh again the same thing again this like if you comment this out think this should uh stop working right

Claudiu Leoveanu-Condrei: Yeah. Yeah,

Andrei Ungureanu: like

Claudiu Leoveanu-Condrei: he does.

Andrei Ungureanu: yeah so it's not working if I save it like this it works again Uh right.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: So uh next topic is this section keyword. This was a bit weird I think.

Claudiu Leoveanu-Condrei: Yeah. This is also to for me it was I I I still don't uh

Andrei Ungureanu: Uh

Claudiu Leoveanu-Condrei: like is is it like a I don't even know I don't I don't have like a mental model of something else. This looks to me like I don't want to call it a cloth, but it's like it's like a I don't know. It's not a cloth actually. That's not right.

Andrei Ungureanu: Yeah,

Claudiu Leoveanu-Condrei: But it's something that Yeah,

00:38:28

Andrei Ungureanu: maybe scope to variables maybe.

Claudiu Leoveanu-Condrei: something. Yeah, something like that. Maybe it's

Andrei Ungureanu: But this is weird because the demo still works.

Claudiu Leoveanu-Condrei: weird.

Andrei Ungureanu: So uh the idea here is that only uh what you define as variables inside the section get dropped. But if you define functions that reference those

variables, they somehow get baked into the function and they still work but the variables uh stop uh existing outside detection. So this is so

Claudiu Leoveanu-Condrei: So it's it's still closure, right?

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: It's like it's like you're in the scope of useful.

Andrei Ungureanu: uh right but uh demo uh function here still exit after the se uh you define this you exit the

Claudiu Leoveanu-Condrei: Ah,

Andrei Ungureanu: section

Claudiu Leoveanu-Condrei: okay. So, this this could be just a way to organize, I

Andrei Ungureanu: maybe to have a sort of like function that

Claudiu Leoveanu-Condrei: assume.

Andrei Ungureanu: you that uses some internal variables

Claudiu Leoveanu-Condrei: Can you do check?

Andrei Ungureanu: or

Claudiu Leoveanu-Condrei: Can you do check on the demo? I hear like what what's the type of

00:39:48

Andrei Ungureanu: logic.

Claudiu Leoveanu-Condrei: that? So it's

Andrei Ungureanu: Oh, so it it references the data which by itself it's an unknown

Claudiu Leoveanu-Condrei: uh

Andrei Ungureanu: identifier at this point because if you do this right so it exit here but doesn't exist here. So it's pretty weird.

Claudiu Leoveanu-Condrei: And if you do check on useful below that, what is the check on useful looking like as a type?

Andrei Ungureanu: So not sure you can do that

Claudiu Leoveanu-Condrei: Oh, sorry. I think it has to be useful.

Andrei Ungureanu: or

Claudiu Leoveanu-Condrei: Yeah. Uh yeah, I think you have to delete the above because it's not defined.

Andrei Ungureanu: yeah just put it.

Claudiu Leoveanu-Condrei: commented. Sorry.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Wait,

Andrei Ungureanu: So yeah,

Claudiu Leoveanu-Condrei: what?

Andrei Ungureanu: cuz it uh I don't think you can

Claudiu Leoveanu-Condrei: But how are you using that?

Andrei Ungureanu: I don't think you can check a section name, right? cuz it

Claudiu Leoveanu-Condrei: Can you do you're you're on Vim, right?

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Can you do shift K on the section on the section

00:41:16

Andrei Ungureanu: Uh here.

Claudiu Leoveanu-Condrei: keyword?

Andrei Ungureanu: Oh, the

Claudiu Leoveanu-Condrei: On the section keyword,

Andrei Ungureanu: section.

Claudiu Leoveanu-Condrei: the limit is the scope of variable include open set option and local local commands. Section can be nested. Section ID provides a label to the section that has to appear with the matching end h. That's a very weird way of structuring information. At least for me right now, it's this this resembles you know what this this resembles to

Andrei Ungureanu: What do you

Claudiu Leoveanu-Condrei: me um I don't know so if you're familiar like with latte like if you put tags in latex when you do slash back slash tag so you're tagging an equation here you're basically giving the section an ID is

Andrei Ungureanu: Mhm. But that ID is non-existent. I mean,

Claudiu Leoveanu-Condrei: I think it it it exists.

Andrei Ungureanu: how do you what's the ID?

Claudiu Leoveanu-Condrei: It's just zero right here.

Andrei Ungureanu: Or maybe you you need another keyword than check. I mean, what do you want to check here?

Claudiu Leoveanu-Condrei: You

00:42:31

Andrei Ungureanu: Cuz there's nothing to check, right? Did you check on functions or types or no

Claudiu Leoveanu-Condrei: agree. I I was I was curious if there is like u if if there's any useful information in that variable but it's not. So you cannot check um yeah I mean it probably because you can have so many like uh definitions.

Andrei Ungureanu: stuff here.

Claudiu Leoveanu-Condrei: Yeah, exactly. So there was the there's no type.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: It's just like it's just a way to write scope. I assume it's just a way to organize uh your code.

Andrei Ungureanu: Yeah.

Nelson Higuera Ruiz: Yes.

Andrei Ungureanu: But this is like the most uh weird way cuz there are also

Nelson Higuera Ruiz: Yes.

Claudiu Leoveanu-Condrei: Exactly. Yeah. I have

Andrei Ungureanu: like normal ways to define

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: this with name spaces. So you basically define a name space and you put everything you want there and you can access it like with the

Claudiu Leoveanu-Condrei: So this is like a class.

Andrei Ungureanu: structure.

00:43:39

Claudiu Leoveanu-Condrei: Yeah. Like a structure. Yeah.

Andrei Ungureanu: Yeah.

Nelson Higuera Ruiz: Mhm.

Andrei Ungureanu: So

Claudiu Leoveanu-Condrei: But can you have sections in the name space?

Andrei Ungureanu: um

Claudiu Leoveanu-Condrei: So if you move that code above, if you copy it and put it into the name space,

Andrei Ungureanu: I

Claudiu Leoveanu-Condrei: will it will it work? It works.

Andrei Ungureanu: guess this right uh wait no I didn't save I wanted to do this so and useful it's okay right code.

Claudiu Leoveanu-Condrei: It works.

Andrei Ungureanu: No, I saved it. Yeah, it works. Yeah,

Claudiu Leoveanu-Condrei: What does it mean?

Andrei Ungureanu: you can uh the the book.

Claudiu Leoveanu-Condrei: Yeah. Yeah. What the hell does it mean to have Let me Yeah.

Andrei Ungureanu: Sorry.

Claudiu Leoveanu-Condrei: Just copy this. I I'm curious to ask Claude. What the hell is this?

Nelson Higuera Ruiz: This doesn't look very useful. I think this maybe I think that you it

Claudiu Leoveanu-Condrei: You you you you said it doesn't or it does. Yeah, I know.

00:44:42

Claudiu Leoveanu-Condrei: I I don't get

Nelson Higuera Ruiz: doesn't

Andrei Ungureanu: But this this actually should be here like un

Claudiu Leoveanu-Condrei: it.

Andrei Ungureanu: useful because whatever this uh auto correction kills me. Um but we we I can here just like

Claudiu Leoveanu-Condrei: Just as an example too.

Andrei Ungureanu: this.

Claudiu Leoveanu-Condrei: Yeah, it is possible.

Andrei Ungureanu: Yeah. Yeah. So you can uh also do this like you can use the open keyword and you open the name space and you don't have to uh use the name space name before accessing it the functions or variables.

Claudiu Leoveanu-Condrei: Do you have a close there?

Andrei Ungureanu: No no it just it stays open like forever in the

Claudiu Leoveanu-Condrei: Okay, that's it's like it's like you're importing

Andrei Ungureanu: file I guess.

Claudiu Leoveanu-Condrei: star or whatever from from it.

Andrei Ungureanu: Yeah, exactly. It's an import.

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: But you can do this stuff uh like uh continue to write uh you

Nelson Higuera Ruiz: Thanks.

Andrei Ungureanu: write some function for this name space and you uh can open like the name space and add some function afterwards and this works right um I I don't know why you would do this cuz it doesn't really make any sense because you would put right all the code in the same place.

00:46:11

Andrei Ungureanu: But I guess it's you can do this. Maybe like we talked before, there's some point to this down the line.

Claudiu Leoveanu-Condrei: Yeah, I mean we're still learning syntax and um yeah,

Andrei Ungureanu: Yeah,

Claudiu Leoveanu-Condrei: I agree.

Andrei Ungureanu: this this stuff usually clicks at some point like it clicked for me with the functions there.

Claudiu Leoveanu-Condrei: I

Andrei Ungureanu: So yeah, um we're learning like this uh for now and we'll see.

Claudiu Leoveanu-Condrei: agree.

Andrei Ungureanu: Uh yes, I think this pretty much it, right? Uh this is the end of.7 to finish. You available sections name spaces.

Claudiu Leoveanu-Condrei: Yeah, that that's it.

Andrei Ungureanu: Yeah. And this is And yeah,

Claudiu Leoveanu-Condrei: Yes,

Andrei Ungureanu: I guess this is bit more abstract.

Claudiu Leoveanu-Condrei: this is more philosophy. I I read a bit from it.

Andrei Ungureanu: Yeah,

Claudiu Leoveanu-Condrei: It's just like more um yeah,

Andrei Ungureanu: that

Claudiu Leoveanu-Condrei: it also has that at which is I I mean show sorry we we I think we even touched this

Andrei Ungureanu: is

Claudiu Leoveanu-Condrei: the last time. Um that that was um how it was called I don't remember right now.

00:47:29

Claudiu Leoveanu-Condrei: Um but but that's in the type it appears. It's like the u type the uh so when you see the at is basically that um it's a cross it's invariant right so when you have a list then the list can essentially take whatever if you if you have strings if you have integers if you have type ones type threes type fours it doesn't matter what it is it's still going to be a list of those things and that's how you mark it with an at that's what it means But we yeah we kind of touched this a bit in the first part like uh of

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: the beaten

Andrei Ungureanu: See universe here as well.

Claudiu Leoveanu-Condrei: path.

Andrei Ungureanu: Whatever that is. Yeah. In the

Claudiu Leoveanu-Condrei: The univer the universe is that thing that uh it's basically the

Andrei Ungureanu: book.

Claudiu Leoveanu-Condrei: so do you remember the sword discussion that we had that long loop uh down

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: the rabbit hole. So the universe U is just like the it's all it's

00:48:28

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: all so there are it's all types literally there is all all the types possible. Uh I think it's down a bit it's towards the

Andrei Ungureanu: Not settle.

Claudiu Leoveanu-Condrei: end.

Andrei Ungureanu: It is. Isn't this what you meant? Or some other text?

Claudiu Leoveanu-Condrei: Uh no no it's it's

Andrei Ungureanu: Oh, here. Right.

Claudiu Leoveanu-Condrei: yeah so yeah exactly so yeah that's it. So you have the type U which is the level it's exact it's like a hierarchy of types. So you have the universe of propositions. So the things that are true or false. Then you have the universe of small data types which is basically natural booleans integer floats whatever uh strings and so on. Then you have the bigger types which is just like abstract types that you define. And then the level U is like literally the it's like abstracting over all the others. That that's what it is. So that's the universe.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: So basically the there there are two universes.

00:49:43

Claudiu Leoveanu-Condrei: It's you envy. So they they declared two universes and they again did the reinforcement learning on you because the list that's why they showed you the list above because the list is a universal type. It is applied to anything which which kind of makes sense right because a list is a list no matter what is its elements it's still a list after like right it's like it doesn't matter

Andrei Ungureanu: All right.

Claudiu Leoveanu-Condrei: there is a list of apples or or what or universes yeah

Andrei Ungureanu: or

Claudiu Leoveanu-Condrei: it's still a list of those things.

Andrei Ungureanu: universes.

Claudiu Leoveanu-Condrei: So that's what's what's a so a universe is just a way to describe uh like general types. So a type that applies to anything like a list. So you're basically defining something that looks conceptually similar to a list. So that whenever you apply that you know that it should be uh it's in a way if you think about it it's like uh it's type agnostic right so you literally abstracted over the whole types all of them like like the type is irrelevant for the list this is how I think about it so there you're defining a function f

00:51:01

Andrei Ungureanu: All right.

Claudiu Leoveanu-Condrei: which is basically invariant to the input type. Actually,

Andrei Ungureanu: But where does the list live then?

Claudiu Leoveanu-Condrei: sorry.

Andrei Ungureanu: And if it's like it's it can contain universes, it has to live like somewhere outside these universes.

Claudiu Leoveanu-Condrei: I mean, if you if you go check list, I mean, is there right?

Andrei Ungureanu: No.

Claudiu Leoveanu-Condrei: If you check list, it's uh I mean, just go in in Lynn and do a checklist. to to just for us to see the the the type because it should be universe plus one or something like that, right? Because the the type in which the universe lives is always plus one. So it has to be something with plus one.

Nelson Higuera Ruiz: It's the universe.

Claudiu Leoveanu-Condrei: Yeah.

Nelson Higuera Ruiz: Seems

Claudiu Leoveanu-Condrei: You see? So it's a list. So it's a type U to type U . You see it's a mapping. It's a mapping that takes α which is of type U . So basically it can be anything and maps it to a type U which is again anything.

00:52:14

Claudiu Leoveanu-Condrei: Now this is very important because on the right you see they declare universe U and V . So basically you can have types that they both share share the same like abstraction but they are still different. It's like a list and a set, right? They they both abstract over all the types but they are still different from one another. Right?

Andrei Ungureanu: Right.

Claudiu Leoveanu-Condrei: That makes sense.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: So you so then you can so then you this allows you then to create mappings of the type a list to a

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: set which yeah I mean it's it's quite clever but I I still don't like fully get it like intuitively you know I I get it because I I manage to to some extent explain it but I don't feel it like I it's too abstract for my brain. Um yeah maybe when we when we have something that deals with proofs and

Nelson Higuera Ruiz: Yes.

Claudiu Leoveanu-Condrei: stuff like this then uh I I would assume then as you say then it should be

00:53:21

Nelson Higuera Ruiz: Yes. Yes.

Claudiu Leoveanu-Condrei: clicking but uh yeah so far it's just syntax but to me it makes sense this this universe and when I when I spend I don't know like four hours or three hours reading about why the hell is it called sort because I thought it's you know it's coming from sorting but it's not it's has nothing

Nelson Higuera Ruiz: Yes,

Claudiu Leoveanu-Condrei: to do with that obviously yeah it's like it's like

Nelson Higuera Ruiz: escort is a synonym of type

Claudiu Leoveanu-Condrei: nothing many sorted logic right it was called yeah many s I will never forget that s***

Nelson Higuera Ruiz: any type logic. Yes.

Claudiu Leoveanu-Condrei: from now on yeah so I spent three hours digging into like but why is it

Nelson Higuera Ruiz: Yes.

Claudiu Leoveanu-Condrei: called like this but why that but why not this like wait what so yeah cool very nice uh then I I would guess that this the left of this chapter is just like we can spend the time to you know uh yeah I mean it it went really really smooth so

probably we could have done it anyways uh right now today uh but we can cover it right on

00:54:31

Andrei Ungureanu: the last chapters.

Claudiu Leoveanu-Condrei: Yeah. Yeah.

Andrei Ungureanu: You mean it seems

Claudiu Leoveanu-Condrei: I mean it was Yeah. I don't know. Um,

Andrei Ungureanu: um a bit dense though? Uh it but if again if

Claudiu Leoveanu-Condrei: it it is it is because I agree with Ch because that's why we didn't do it

Andrei Ungureanu: it Oh,

Claudiu Leoveanu-Condrei: today.

Andrei Ungureanu: right. Right.

Claudiu Leoveanu-Condrei: It's fair.

Andrei Ungureanu: Right.

Claudiu Leoveanu-Condrei: It's just like it's it's it's shorter but it's it's dense because it it deals with this obstruction which probably is going to going to break our brains. Uh but uh yeah,

Andrei Ungureanu: Mhm.

Claudiu Leoveanu-Condrei: I mean it it went really smooth in my opinion

Nelson Higuera Ruiz: Yes, for sure. I I like it.

Claudiu Leoveanu-Condrei: today.

Andrei Ungureanu: Yeah.

Nelson Higuera Ruiz: Thank you.

Andrei Ungureanu: Yeah.

Nelson Higuera Ruiz: Thank you, Andre. It was great.

Andrei Ungureanu: No worries. My pleasure.

Claudiu Leoveanu-Condrei: Very nice.

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: Uh yeah. Uh I suppose you can um can you give me your GitHub handle so that you can push this uh your thing to the repo because it's it's easier

00:55:27

Andrei Ungureanu: Yeah. Yeah, sure. Uh, I'll write it in the

Claudiu Leoveanu-Condrei: than Yeah. Yeah. Yeah. Put it here. Yeah. Because I Yeah,

Andrei Ungureanu: Discord.

Claudiu Leoveanu-Condrei: it's easier for you to just push it than to give it to me and then me pushing it. Very nice. Okay. Uh then like you said you have uh you go into your holiday right the next two weeks.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: Yeah.

Claudiu Leoveanu-Condrei: So how should be doing what should we do then? Should we take a break for two weeks? I mean I like that again you you seem to be into it and I like it. Um and

Andrei Ungureanu: I don't want to be a burden or anything. I mean,

Claudiu Leoveanu-Condrei: yeah

Andrei Ungureanu: it's up to you, I guess. Uh like if you if you want to go forward

Claudiu Leoveanu-Condrei: again you

Andrei Ungureanu: uh I can catch up or I don't know maybe try to

Claudiu Leoveanu-Condrei: maybe we can do this last chapter like Nelson and

00:56:28

Andrei Ungureanu: uh

Claudiu Leoveanu-Condrei: me um and then and then

Nelson Higuera Ruiz: and then wait for you to come back have

Claudiu Leoveanu-Condrei: yeah then we can start the chapter three

Nelson Higuera Ruiz: any.

Claudiu Leoveanu-Condrei: together.

Andrei Ungureanu: Okay. And then I can look at these two chapters this week where I live.

Claudiu Leoveanu-Condrei: Yeah, you can look yourself in the two weeks.

Andrei Ungureanu: And yeah.

Claudiu Leoveanu-Condrei: Yeah.

Andrei Ungureanu: Yeah, it's it's good this

Claudiu Leoveanu-Condrei: Okay, perfect. Cool.

Nelson Higuera Ruiz: Okay.

Claudiu Leoveanu-Condrei: Then Nelson,

Andrei Ungureanu: week.

Claudiu Leoveanu-Condrei: is it fine with you right with uh with like this

Nelson Higuera Ruiz: Yeah,

Claudiu Leoveanu-Condrei: hour?

Nelson Higuera Ruiz: I think that is working. I mean, if I have a problem, I will let you know anyhow. sometimes, but for now seems seems perfect to me

Claudiu Leoveanu-Condrei: Okay. So yeah, we we kind of move like once a week because we yeah,

Nelson Higuera Ruiz: actually.

Claudiu Leoveanu-Condrei: I mean we cover ground, we stay warm and uh yeah, we yeah, we move on. So good.

00:57:23

Claudiu Leoveanu-Condrei: Then let's do it like this. Then the next time um do you want to be the guide or I can be the guide? I'm fine. As you wish.

Nelson Higuera Ruiz: Good question. Yeah, I also want to start doing something but maybe maybe I will we'll we'll wait for next one. Maybe you can guide the next one please.

Claudiu Leoveanu-Condrei: Okay, cool. Then I then I'm going to do it then.

Nelson Higuera Ruiz: Thank you. Thank

Claudiu Leoveanu-Condrei: Okay,

Nelson Higuera Ruiz: you.

Claudiu Leoveanu-Condrei: then let me invite Andre to the

Andrei Ungureanu: So you want to do uh the next two weeks uh another

Claudiu Leoveanu-Condrei: repo.

Andrei Ungureanu: um session like

Claudiu Leoveanu-Condrei: Yeah,

Andrei Ungureanu: this.

Claudiu Leoveanu-Condrei: I'm going to do it next week uh for the last two chapters.

Andrei Ungureanu: Okay. So uh on the 18th, right?

Claudiu Leoveanu-Condrei: Uh yes, same

Andrei Ungureanu: That would be okay. Yeah,

Claudiu Leoveanu-Condrei: order.

Andrei Ungureanu: I'll try to to uh attend as well. Uh it's going to be my first day with flights and stuff, so not sure if I can make it, but I'll try to make it.

00:58:25

Claudiu Leoveanu-Condrei: Best time to do it, man. In the airport. That's cool.

Andrei Ungureanu: Uh

Claudiu Leoveanu-Condrei: That's going to be amazing. The the best place to wear headphones.

Andrei Ungureanu: yeah. Yeah.

Claudiu Leoveanu-Condrei: Nice. Okay. Uh let me invite it to the ripple. Okay. Then you you will see the structure how how is it in the like how Yeah. I mean you will see but why c can I get it the the handle alone not

Andrei Ungureanu: Oh, the handle, right? It's the same

Claudiu Leoveanu-Condrei: email?

Andrei Ungureanu: as

Claudiu Leoveanu-Condrei: Okay. Yeah. You added you should have got it. uh for whatever reason I don't I cannot give you right now the rights because I think you have to accept first but once you do I I will give you like yeah full

Andrei Ungureanu: So, was it supposed to come on the email or just in uh GitHub a

Claudiu Leoveanu-Condrei: rights I think it's

Andrei Ungureanu: notification?

Claudiu Leoveanu-Condrei: on your

Andrei Ungureanu: Oh, it's a email,

00:59:47

Claudiu Leoveanu-Condrei: mail.

Andrei Ungureanu: right? Yeah.

Claudiu Leoveanu-Condrei: Okay, cool then. Yeah. Uh,

Andrei Ungureanu: Yeah, accept it. So,

Claudiu Leoveanu-Condrei: thank you.

Andrei Ungureanu: I can see it now. I think I'm

Claudiu Leoveanu-Condrei: Let me see what you But why can I not give you

Andrei Ungureanu: in

Claudiu Leoveanu-Condrei: rights? Wait, what?

Andrei Ungureanu: there. You have it.

Claudiu Leoveanu-Condrei: It's like I mean I hope you can push

Andrei Ungureanu: You don't. No. All right. I don't have access to the settings tab.

Claudiu Leoveanu-Condrei: I

Andrei Ungureanu: So, I can try to push to see if this

Claudiu Leoveanu-Condrei: there's no there's it doesn't appear like to give you it's like you it's

Andrei Ungureanu: works.

Claudiu Leoveanu-Condrei: like in showing you have access like you you can do whatever you

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: Let me check with N.

Nelson Higuera Ruiz: Mhm.

Claudiu Leoveanu-Condrei: No, it's like the same for him as well. So I assume you have the rights to do that. It's inferred probably automatically.

Andrei Ungureanu: Okay.

01:01:10

Claudiu Leoveanu-Condrei: Yeah, there's nothing I I have to check on my end. Yeah, just try and if it doesn't work then you you let me know and

Andrei Ungureanu: Okay. Okay.

Claudiu Leoveanu-Condrei: ah I have to oh I that that I would add myself. So the session I will uh the notes from this call I will add them myself. You can just add the the read me and

Andrei Ungureanu: So just push to branch and you do the merge or

Claudiu Leoveanu-Condrei: the no just make a new

Andrei Ungureanu: do Yeah,

Claudiu Leoveanu-Condrei: session underscore003 then I mean you you see

Andrei Ungureanu: that's what I did.

Claudiu Leoveanu-Condrei: the yeah push domain

Andrei Ungureanu: But uh do I push to main or to a different branch that Oh,

Claudiu Leoveanu-Condrei: man it's fine we don't need to do branches and all of that it's not

Andrei Ungureanu: okay. Okay. cuz I changed the I mean not change I I uh put some

Claudiu Leoveanu-Condrei: Like

Andrei Ungureanu: extra uh text in the session 002 so maybe like doesn't make sense

Claudiu Leoveanu-Condrei: but it but it's but it's good if you if you are if you if you have

01:02:08

Andrei Ungureanu: for yeah just it's

Claudiu Leoveanu-Condrei: that

Andrei Ungureanu: but mostly for me but I'm not sure no it's you know it's if

Claudiu Leoveanu-Condrei: ah I see then I I

Andrei Ungureanu: you you go through it then maybe you'll see what the hell is this like when did Right. Like this.

Claudiu Leoveanu-Condrei: No, then just let this file that you guide us

Andrei Ungureanu: This. Okay.

Claudiu Leoveanu-Condrei: through.

Andrei Ungureanu: Okay. It's better like this. Yeah.

Claudiu Leoveanu-Condrei: Okay.

Andrei Ungureanu: Right. Are you guys still there?

Claudiu Leoveanu-Condrei: Uh,

Nelson Higuera Ruiz: Yes.

Andrei Ungureanu: Okay.

Claudiu Leoveanu-Condrei: yes.

Andrei Ungureanu: Was a silence. I didn't know what happened.

Claudiu Leoveanu-Condrei: No,

Nelson Higuera Ruiz: No.

Claudiu Leoveanu-Condrei: we're here. So, yeah. Do you want to push now or do you want to clean it?

Andrei Ungureanu: Yeah, I want to clean it a bit. So,

Claudiu Leoveanu-Condrei: Oh,

Andrei Ungureanu: uh I'll push it like in the next 10 minutes or

Claudiu Leoveanu-Condrei: okay. Okay, then if it doesn't work,
Andrei Ungureanu: so.
Claudiu Leoveanu-Condrei: then ping me on Discord and I will look
Andrei Ungureanu: Oh,
Claudiu Leoveanu-Condrei: again.
Andrei Ungureanu: okay. Okay.
Claudiu Leoveanu-Condrei: Okay, cool. Then thank you, man. And uh see you guys next
Andrei Ungureanu: Sure.
Claudiu Leoveanu-Condrei: week.
Nelson Higuera Ruiz: All right.
Andrei Ungureanu: Yes.
Nelson Higuera Ruiz: Thank you.
Andrei Ungureanu: Hopefully.
Nelson Higuera Ruiz: Hopefully.
Andrei Ungureanu: Yes.
Nelson Higuera Ruiz: Thank you,
Claudiu Leoveanu-Condrei: Yeah.
Nelson Higuera Ruiz: Andre. And see you next week. Byebye.
Claudiu Leoveanu-Condrei: See you.
Andrei Ungureanu: Yeah. Thank you.
Claudiu Leoveanu-Condrei: Bye-bye.
Andrei Ungureanu: Bye-bye. Bye.

Transcription ended after 01:03:46

This editable transcript was computer generated and might contain errors. People can also change the text after it was created.